

Multi-Agent AI System Architecture

Master Technical Guide & Step-by-Step Execution Walkthrough for Backend Engineers

1. Executive System Overview

A Multi-Agent System breaks down complex tasks into autonomous, single-responsibility components. Unlike traditional monolithic scripts, each phase utilizes a dedicated LLM instance combined with specific tools or prompt pipelines to search, scrape, format, and audit research reports.

Phase 0: Router	Supervisor LLM reads user query and determines the execution sequence.
Phase 1: Search	Search Agent calls <code>web_search()</code> tool via Tavily API to gather links.
Phase 2: Scrape	Scrape Agent calls <code>scrape_url()</code> tool to extract clean page text.
Phase 3: Writer	Writer Chain formats raw research into a structured Markdown report.
Phase 4: Critic	Critic Chain audits report quality, providing strengths, areas to improve, and score.

2. Why An LLM Is Required At Every Step

From a backend developer perspective, standard Python code is deterministic, while LLMs act as the reasoning CPU:

Component	Why LLM is Required
Search Agent	Decides search strategy, formulates exact search queries, and evaluates if search results are sufficient.
Scrape Agent	Filters dozens of search result links to select only the most authoritative URLs to scrape.
Writer Chain	Synthesizes thousands of words of messy scraped text into structured, human-quality paragraphs.
Critic Chain	Acts as an independent QA auditor, evaluating facts, structure, and scoring report quality.

3. Concrete Step-by-Step Execution Walkthrough

Example Query: *"Virat Kohli performance in the recent year"*

Step 1: User Query Submitted

User submits request string: 'Virat Kohli performance in the recent year'.

Step 2: Search Agent LLM Formulates Query

Search Agent receives user prompt, reasons about intent, and triggers `web_search` tool:

```
web_search.invoke({'query': 'Virat Kohli match performance stats recent year'})
```

Step 3: `web_search` Tool Payload Output

Tavily API executes search and returns title, URL, and snippet payload:

Title: ESPNcricinfo - Virat Kohli Stats
URL: <https://www.espnccricinfo.com/player/virat-kohli-253802>
Snippet: Scored 1,042 runs in recent matches with an average of 52.1...

Step 4: Scrape Agent Selects URL & Invokes Scraper

Scrape Agent evaluates links and triggers `scrape_url` on ESPNcricinfo:

```
scrape_url.invoke({'url': 'https://www.espnccricinfo.com/player/virat-kohli-253802'})
```

Step 5: `scrape_url` Tool Returns Cleaned Web Text

BeautifulSoup strips boilerplate (header/footer/nav) and `textwrap` formats text:

```
Virat Kohli Performance Summary:  
Total Matches: 18 | Innings: 24 | Runs: 1,042 | Average: 52.1  
Centuries: 3 | Half-Centuries: 5 | Top Score: 138 vs Australia
```

Step 6: Writer Chain Synthesizes Markdown Report

Writer Prompt injects topic + scraped research, and LLM generates report:

```
# Research Report: Virat Kohli Recent Performance  
  
## Key Findings  
1. Heavy Run Scoring: 1,042 runs across 18 matches at 52.1 average.  
2. Match Winning Centuries: 3 centuries including 138 vs Australia.  
  
## Sources  
- https://www.espnccricinfo.com/player/virat-kohli-253802
```

Step 7: Critic Chain Audits Report Quality

Critic Prompt receives report and LLM generates structured evaluation:

```
Score: 9/10  
  
Strengths:  
- Accurate match statistics and cited source link.  
  
One line verdict: Excellent fact-backed performance report.
```

4. Orchestration: Who Decides Execution Order?

1. Pipeline Script (Developer Hardcoded): The developer writes a main script (`main.py`) that passes data sequentially: Search -> Scrape -> Writer -> Critic.

2. Supervisor Agent (Dynamic Router): A central Manager LLM inspects agent descriptions and routes user requests dynamically based on context.